CMP 519 Software Engineering

UNIT 08 – Emerging Trends in Software Engineering

Instructor: Dr. Suresh Pokharel

DISCLAIMER

These slides are part of teaching materials for Software Engineering. These slides do not cover all aspect of learning Software Engineering, nor are these be taken as primary source of information. As the core textbooks and reference books for learning the subject has already been specified and provided to the students, students are encouraged to learn from the original sources.

Cloud-Native Development

Microservices Architecture

- Principles:
  - *Decentralization*: Each service is developed, deployed, and scaled independently.
  - *Single Responsibility*: Each service focuses on a single business capability.
  - *Automation:* Emphasizes CI/CD pipelines for automated testing and deployment.
  - *Resilience*: Services are designed to handle failures gracefully.

- Benefits:
  - *Scalability*: Services can be scaled independently based on demand.
  - *Flexibility*: Different technologies can be used for different services.
  - *Faster Time-to-Market*: Independent deployment allows for quicker updates and releases.
  - *Improved Fault Isolation*: Issues in one service do not necessarily affect others.

- *Example*: Netflix uses microservices to manage user authentication, recommendations, and video streaming.

Microservices Architecture

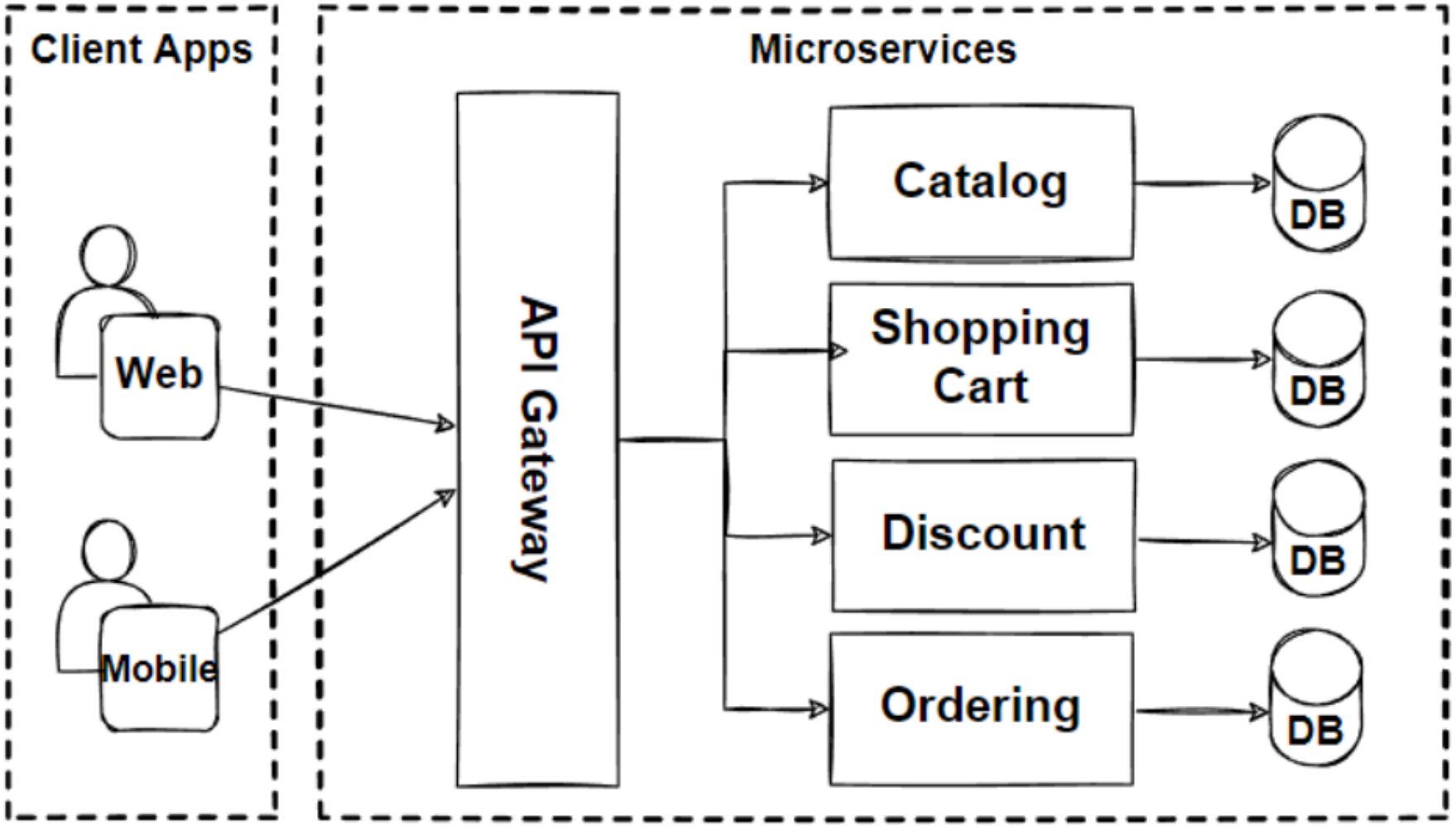
Microservices Architecture

- Challenges:

  - Complexity: Managing multiple services can be complex.
  - Data Consistency: Ensuring consistency across services is challenging.
  - Latency: Inter-service communication can introduce delays.

- Tools and Technologies:
  - Docker (Containerization)
  - Kubernetes (Orchestration)
  - Spring Boot (Java Microservices)
  - Istio (Service Mesh)

Microservices Architecture



An example of microservices architecture

Serverless Computing

- Concepts:
  - Event-Driven: Functions execute in response to events.
  - Stateless: Each function execution is independent.
  - Auto-Scaling: The platform scales functions dynamically.
- Use Cases:
  - Real-time file processing
  - API and microservices deployment
  - IoT data processing
- Advantages:
  - Cost-effective (pay for execution time only)
  - Automatic scaling
  - Reduced operational overhead
- Limitations:
  - Cold start latency
  - Vendor lock-in
- Tools and Technologies:
  - AWS Lambda
  - Azure Functions
  - Google Cloud Functions

Serverless Computing

An example of serverless computing

https://www.geeksforgeeks.org/serverless-architectures/

AI in Software Engineering

AI-Assisted coding and Testing

- Tools:
  - GitHub Copilot (AI-powered code completion)
  - Kite (AI coding assistant)
  - DeepCode (AI-based code review)
- Impact on Development:
  - Increased productivity
  - Improved code quality
  - Faster onboarding
- Challenges:
  - Over-reliance on AI
  - Security concerns
  - Limited contextual understanding
- Machine Learning Applications
  - Predictive maintenance
  - Recommendation systems
  - Natural Language Processing (NLP)
- Challenges:
  - Data quality issues
  - Model interpretability
  - Ethical concerns

Software for IoT and Edge Computing

- Challenges:
  - Resource constraints
  - Network latency
  - Security vulnerabilities
- Best Practices:
  - Optimize code
  - Use edge computing for real-time processing
  - Implement security by design
- Tools and Technologies:
  - MQTT (Messaging Protocol)
  - EdgeX Foundry (Open-source edge computing platform)
  - TensorFlow Lite (ML for edge devices)

Sustainability in Software Engineering

Eco-Friendly Practices

- Techniques:
  - Optimize algorithms
  - Reduce redundancy
  - Use asynchronous processing
- Benefits:
  - Reduced energy consumption
  - Extended battery life
  - Lower environmental impact
- Challenges:
  - Performance trade-offs
  - Complexity in implementation

Secure Software Development Practices

Basic Principles of Writing Secure Code

- Principles:
  - Input validation
  - Least privilege access
  - Secure communication (Encryption)
- Common Vulnerabilities and Mitigation:
  - SQL Injection: Use parameterized queries.
  - XSS: Sanitize user input.
  - Buffer Overflow: Implement bounds checking.
- Tools and Technologies:
  - OWASP ZAP (Security Testing)
  - SonarQube (Code Analysis)
  - Burp Suite (Web Security Testing)

Integrating Security Testing into Development

- Strategies:
  - Static analysis
  - Dynamic analysis
  - Penetration testing

END OF UNIT 08